PREMIER UNIVERSITY, CHITTAGONG

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FINAL PROJECT REPORT

Citizen Management

Field	Detail
Project Name	Citizen Management
Course Name	Software Development Project
Course Code	CSE 3000
Semester	6th
Section	E
Batch	44th
Session	Spring 2026
Date of Submission	29.09.26

Submitted By,

Name	ID
Sazib Hossain	0222320005101174
Mohammad Rayan	0222320005101186
Ifty Anwar	0222320005101192
Santusti Biswas	0222320005101194

Submitted To,
Rashed Miah
Lecturer
Department of Computer Science and Engineering
Premier University Chattogram

Table of Contents

1. Cover Page 1
2. Table of Contents 2
3. Abstract 3
4. Introduction 3
5. Literature Review / Related Work 7
6. Methodology 7
7. System Design and Architecture 10
8. Implementation 21
9. Results and Discussion 23
10. Conclusion and Future Work 24
Appendix 25

Abstract

The Citizen Management system is a web-based municipal service platform built to digitalize citizen-facing operations such as identity verification, property record-keeping, rental agreement management, property tax collection, and complaint handling for organizations such as the Chattogram City Corporation. The system replaces fragmented, paper-driven workflows with a single, role-based portal so that citizens and administrators interact through consistent, auditable digital processes.

Following the design phase, the system was implemented as a native ASP.NET Core MVC application (.NET 10, C#) backed by PostgreSQL through Entity Framework Core, with cookie-based authentication and role authorization, Razor views for the presentation layer, and an optional Stripe.NET integration for online tax payment. This report documents the final, as-built system: its requirements, architecture, module design, implementation, and evaluation.

The finished system supports six core modules — Authentication, Citizen (NID) Verification, Property Management, Rent Agreement Management, Tax Payment, and Complaint Management — developed using a hybrid Agile–Spiral process. The result is a modular, role-secured, and extensible platform that demonstrates end-to-end municipal service digitalization and provides a foundation for future enhancements such as national ID system integration, mobile access, and analytics.

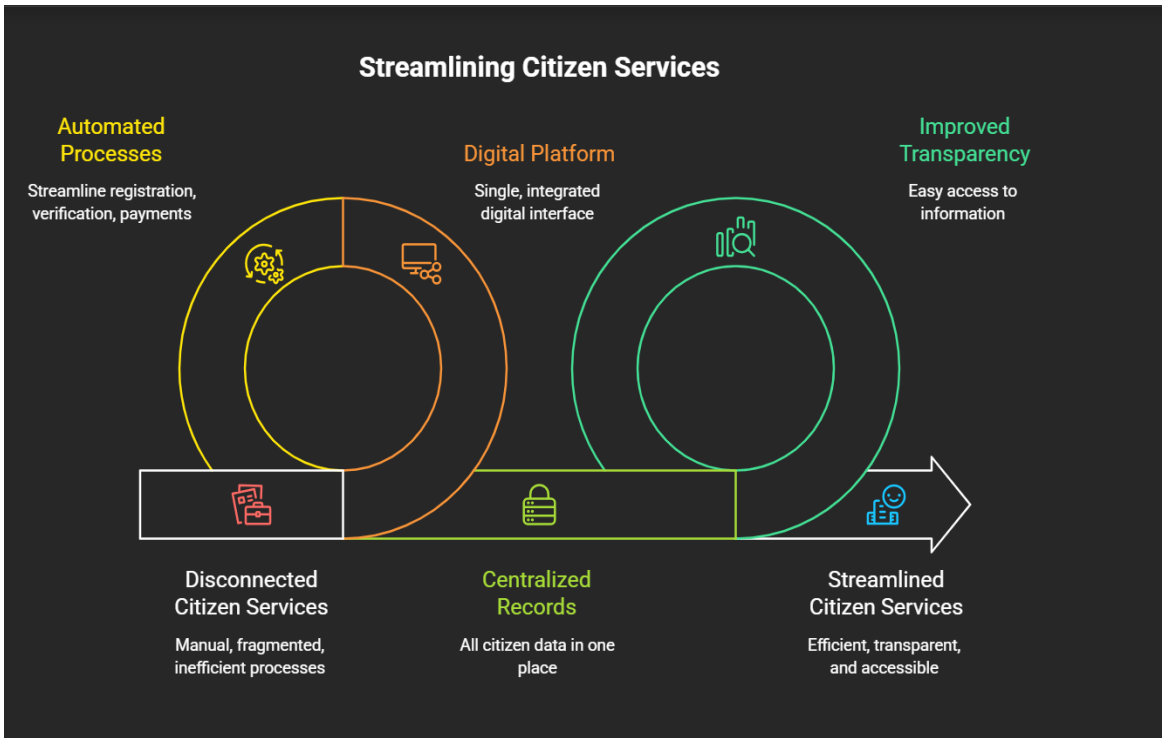
Introduction

Background and Motivation

Urban municipalities in Bangladesh, including city corporations such as Chattogram, largely depend on manual, paper-based procedures for citizen verification, property record-keeping, rent documentation, tax collection, and complaint resolution. These manual processes are slow, error-prone, and lack transparency, which reduces citizen trust and increases the administrative burden on municipal staff. The Citizen Management project was undertaken to address this gap by building a centralized digital platform for these services.

Problem Statement

Citizens currently have no single digital channel to register, get identity-verified, manage property records, pay property tax, or lodge complaints; each of these tasks is handled through separate, disconnected, and largely manual office procedures. Administrators similarly lack a unified dashboard to track verification requests, property changes, tax collection, and complaint resolution, which makes oversight and reporting difficult.



Objectives

- Provide secure, role-based authentication for citizens and administrators.
- Digitize citizen identity verification, including a cross-check against a reference national ID (NID) dataset.
- Allow citizens to register, update, and transfer property records subject to administrative approval.
- Digitize rental agreements between property owners and tenants, generated once a rental request is approved.
- Provide an online property tax workflow — assessment issuance, viewing dues, and payment (including a Stripe-based online payment option).
- Provide a structured complaint submission, tracking, and resolution channel.

Scope

The delivered system covers six modules: Authentication, Citizen Verification, Property Management, Rent Agreement, Tax Payment, and Complaint Management, implemented as a single ASP.NET Core MVC web application with a PostgreSQL database. The scope for this academic project is limited to a single municipal entity model with two roles (citizen and administrator); production-grade integrations such as a live national ID registry, SMS gateways, or a production payment gateway are represented in demo/mock form.

System Requirements Specification (SRS) — from the Approved Project Proposal

The functional requirements below are carried over unchanged from the project proposal submitted to the course instructor for the "Citizen Management" project. The right-hand column records each requirement's status in the final ASP.NET Core / PostgreSQL build.

ID	Functional Requirement	Actor	Status in Final Build
FR-AUTH-01	Email-based registration with unique verification status set to pending.	Unverified Citizen	Delivered — AuthController registration; User.VerificationStatus defaults to "pending".
FR-AUTH-02	Role-Based Access Control (RBAC) enforced via custom middleware stacks.	System Controller	Delivered — role-based authorization checks (citizen/admin) enforced at the controller/action level using ASP.NET Core's built-in authorization middleware.
FR-VER-01	Verification of citizen identity through uploaded documents and reference datasets.	Admin / System Backend	Delivered — NID front/back and photo upload, cross-checked against the FakeNid reference dataset, with admin approve/reject.
FR-VER-02	Generation of a downloadable PDF summary document embedded with a tracking QR code.	Verified Citizen	Partially delivered — a static in-browser "Verification Certificate" confirmation page (Views/verification/certificate.cshtml) is shown after approval; it does not yet generate a QR code, a vCard payload, or a downloadable PDF file — the "Download Certificate" button currently only links back to the status page.
FR-PROP-01	Metadata submission capturing location parameters, land-use data, and digital deeds.	Verified Citizen	Delivered — PropertyRequest captures title, address/city, area, land use, assessed value, and rent terms for admin review.

ID	Functional Requirement	Actor	Status in Final Build
FR-PROP-02	Structured ownership transfer request wizard requiring matching multi-party clearance.	Citizen / Admin	Partially delivered — citizen submits a transfer request by the new owner's email and an admin approves it; the proposal's multi-party (both-owner) confirmation step was simplified to single admin approval.
FR-RENT-01	Dynamic template mapping landlord, tenant, and property into legal agreement drafts.	System Core	Delivered — approved RentalRequests generate a RentAgreement with landlord, tenant, property, term, rent, and deposit fields.
FR-TAX-01	Automated fiscal year tax assessment and outstanding balance calculation.	System Cron / Admin	Delivered — TaxAssessmentService generates per-fiscal-year assessments at a configurable rate; OverdueAssessmentService flags unpaid dues.
FR-TAX-02	Integration of a secure mock API/Stripe checkout session for payment handling.	Stripe Checkout	Delivered — StripeCheckoutService (Stripe.NET) with a DemoMode fallback covering Card, Bank Transfer, bKash, and Nagad.
FR-COMP-01	Public grievance submission module providing categorical sorting and explicit tracking references.	Grieved Citizen	Delivered — Complaint model with category, status (open/in review/resolved/rejected), and a numeric ticket ID as the tracking reference.

Non-Functional Requirements (NFR)

- Performance & Latency: standard data rendering, queries, and receipt generation should complete within 3 seconds under normal concurrent load.
- Security & Hashing: sensitive traffic runs over HTTPS; passwords are salted/hashed using ASP.NET Core's built-in IPasswordHasher; authentication routes should be rate-limited against brute-force attempts.
- Data Integrity: financial transaction flows (tax assessments and payments) follow transactional idempotency to protect records against race conditions.
- Cross-Browser Compatibility: the application must function across mainstream browsers (Chrome, Firefox, Safari) and scale responsively to mobile/tablet screens.

Literature Review / Related Work

Similar Applications

E-governance and municipal self-service portals are widely used internationally to replace over-the-counter civic services — examples include property-tax self-service portals run by city corporations, national digital-identity portals, and utility/complaint-management systems used by municipal bodies. These systems share a common pattern: citizen self-registration and identity verification, a searchable record of civic assets (property, licenses, utilities), an online payment channel, and a ticketed complaint/grievance module. The Citizen Management project follows this same pattern at a project scale suitable for a single city corporation, combining property, tax, rent, and complaint handling in one authenticated portal rather than as separate siloed systems.

Key Technologies and Frameworks Evaluated

The project proposal's Technology Stack section identified ASP.NET Core as the backend framework, and the team delivered the implementation on ASP.NET Core MVC with C#, Entity Framework Core, and PostgreSQL. This stack follows the Model–View–Controller (MVC) pattern and offers mature ORM tooling (EF Core), server-rendered templating (Razor), and first-party authentication middleware. The choice of ASP.NET Core MVC (.NET 10) was driven by the team's familiarity with strongly-typed C#, Visual Studio tooling, and EF Core's compile-time model checking, while PostgreSQL was selected for its strong support for structured/relational municipal data and its free, open-source licensing under Npgsql's EF Core provider.

For payments, Stripe was retained as the reference payment gateway (via the official Stripe.NET SDK) because it offers a well-documented hosted checkout flow and a webhook model suitable for demonstrating an end-to-end online-payment workflow without processing real card data during development.

Methodology

Development Approach

Per the project proposal, the team followed a hybrid Agile (Scrum) and Spiral process model. Agile allowed the team to build the platform iteratively in 1–2 week sprints, gathering rapid feedback on frontend dashboards and CRUD logic, one module at a time (Authentication, Verification, Property, Rent Agreement, Tax, Complaint). The Spiral model provided structured risk-analysis intervals to test and secure higher-risk, third-party-facing elements — the Stripe checkout flow, document/PDF handling, and the database connection layer — before they were wired into the main application.

Technology Stack Overview

The table below summarizes the technology stack named in the approved project proposal (Section 7.2) alongside what was actually shipped in the final build (see the accompanying project source archive). The two are consistent at the framework level, with the delivered build finalizing specific library, database, and payment-provider choices during implementation.

Category	Proposal (Section 7.2)	Delivered in Final Build
Backend Framework	ASP.NET Core MVC on C#, per proposal §7.2	ASP.NET Core MVC on C#, .NET 10
Presentation Layer	Razor views + Tailwind-style utility CSS	Razor views (.cshtml) + a custom CSS design system (wwwroot/css), Plus Jakarta Sans font
Database Layer	PostgreSQL (relational)	PostgreSQL (relational), accessed via Entity Framework Core 10 / Npgsql
Auth & Security	Cookie authentication + role-based authorization, salted password hashing	ASP.NET Core Cookie Authentication + role-based authorization; ASP.NET Core IPasswordHasher
Payments	Stripe.NET (mock checkout)	Stripe.NET, with hosted Stripe Checkout and a built-in DemoMode covering Card, Bank Transfer, bKash, and Nagad
Certificates / QR	dompdf (PDF) + qrserver.com API	Static in-browser certificate confirmation page (no QR code or vCard data generated); server-side PDF export not yet implemented
IDE / Tooling	Visual Studio Code	Visual Studio
Version Control	Git & GitHub	Git and GitHub
Design	Figma	Figma (wireframing during the design phase)

Team Roles and Week-wise Contributions (per Project Proposal)

The proposal assigned the eight-week plan across the four team members as follows; this division of labour was carried through into the delivered ASP.NET Core implementation.

Wk	Focus	Sazib Hossain	Mohammad Rayan	Ifty Anwar	Santusti Biswas
1	Requirement synthesis & core architecture	All members: DB normalization, ER design, use cases, architecture modeling, initial project environment setup			
2	Authentication & middleware	Registration routes; secure password hashing	Core login logic and exception controllers	RBAC middleware; email-scope validation	Responsive login/signup UI
3	Identity verification engine	NID parsing logic; verification algorithms	External QR asset generation	Verification queue, approvals, rejection handlers	Citizen document upload UI
4	Property lifecycle management	Asset logs, addition tables, data models	Ownership-transfer pipelines & backend models	Database integration & dataset safety	Searchable asset tables & filter panels
5	Rental agreement drafting	Dynamic template formatting utilities	Property registry & tenant data pipelines	Document generation testing & layout validation	Rental application dashboard forms
6	Tax assessment & billing	Stripe checkout session integration	Tax invoice & balance computation	Webhook listeners & billing tests	Ledger summary & transaction history screens
7	Grievance management	Complaint ticket tracking & categorization	Real-time status updates for admins	Attachment file rules & backend security	Citizen ticket forms & admin analytics grids

Wk	Focus	Sazib Hossain	Mohammad Rayan	Ifty Anwar	Santusti Biswas
8	QA, tuning & release	Refactoring, query optimization, bug tracking	Server/database deployment finalization	Security audits & integration testing	Mobile-responsive fixes & CSS polish

Project Timeline / Milestones

The eight-week schedule below reflects the module-by-module delivery plan used across both the design and implementation phases.

Week(s)	Milestone
1–2	Requirement analysis and system design (SRS, use case, sequence, activity, state, and class diagrams)
2–3	Authentication module: registration, login, role-based access control
3–4	Citizen verification module: NID submission, admin review, certificate issuance
4–5	Property management module: add/update/transfer requests, admin approval, browse/search
5	Rent agreement module: agreement generation from approved rental requests
5–6	Tax payment module: assessment generation, online/demo payment, receipts
6–7	Complaint management module: submission, assignment, resolution
7–8	Integration, manual workflow verification, and final report preparation

System Design and Architecture

High-Level Architecture

The application follows the Model–View–Controller (MVC) architectural pattern, implemented natively with ASP.NET Core MVC. The table below maps each MVC layer to its concrete implementation in the delivered codebase.

Layer	Responsibility	Implementation
View	Presentation / UI, forms, dashboards	Razor views (.cshtml) under Views/, using a shared layout (_Layout.cshtml) and a custom dark-themed CSS design system
Controller	Request handling, routing, authorization checks	15 ASP.NET Core MVC controllers — AuthController, BaseController, PropertyController, AdminController, AdminPropertyController, CitizenController, CitizenTaxController, AdminTaxController, VerificationController, ComplaintController, AdminComplaintController, AdminRentAgreementController, CitizenRentAgreementController, AdminRevenueController, FakeNidStatusController
Service	Business logic shared across controllers	TaxAssessmentService, RevenueDashboardService, StripeCheckoutService, NidVerificationService, OverdueAssessmentService, EmailService
Model / Data	Entities and persistence	EF Core entity classes (User, Property, PropertyRequest, RentalRequest, RentAgreement, TaxAssessment, TaxPayment, Complaint, FakeNid) mapped via ApplicationDbContext to PostgreSQL

Note on the design documentation: the use case, sequence, activity, state, and class diagrams below were produced during the design phase and named after the delivered ASP.NET Core controllers and services (e.g., AuthController, VerificationController, PropertyController, AdminTaxController, CitizenTaxController, StripeCheckoutService, ComplaintController,

AdminRevenueController). The workflow and component structure they describe are implemented consistently throughout the final C# implementation, built on the ASP.NET Core MVC / EF Core / PostgreSQL stack.

Use Case Diagrams

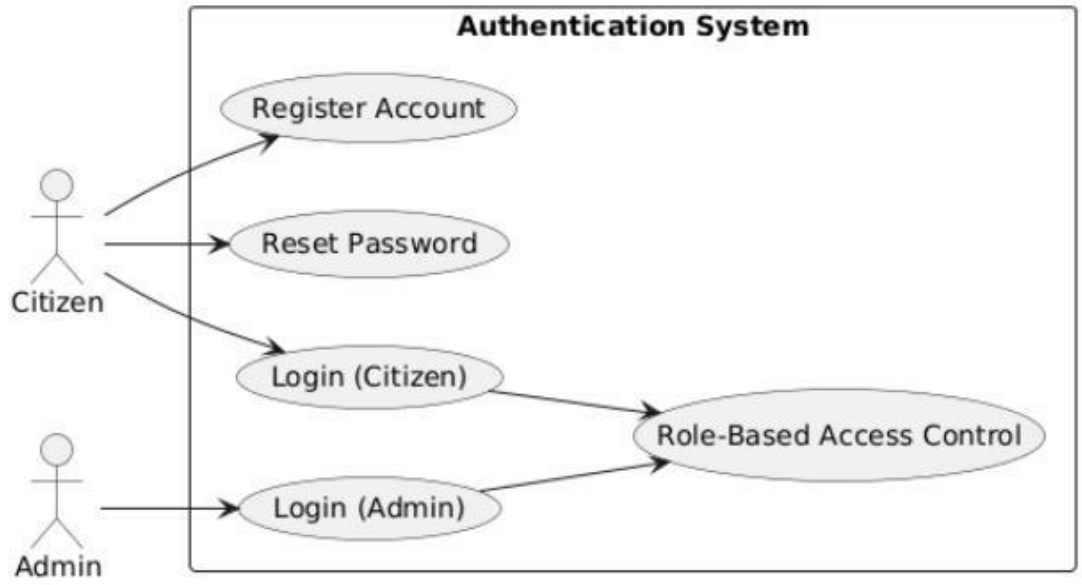


Fig 1: Use case diagram — Authentication module

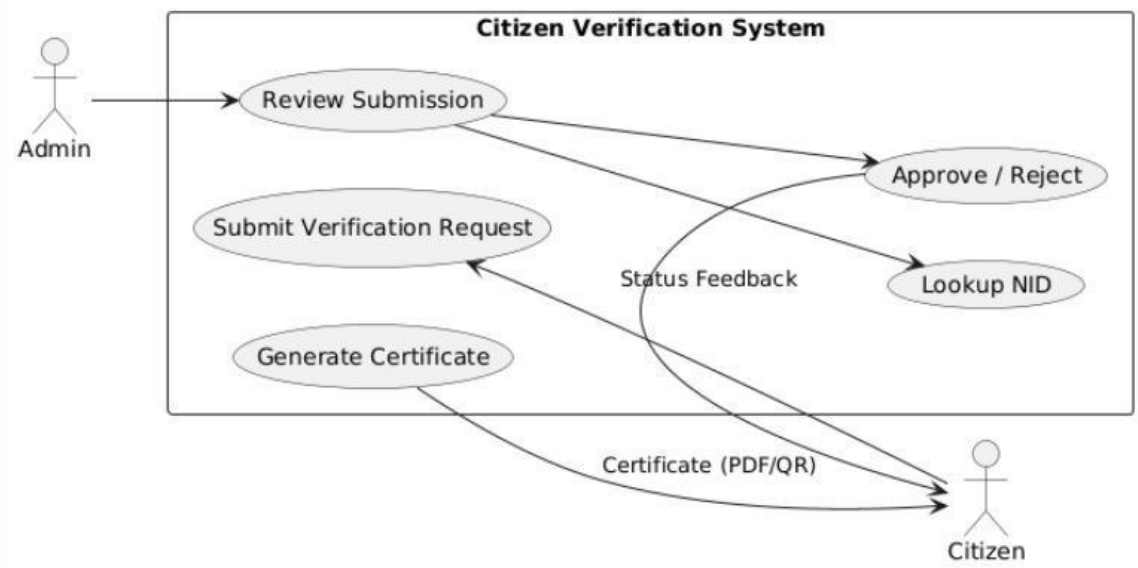


Fig 2: Use case diagram — Citizen Verification module

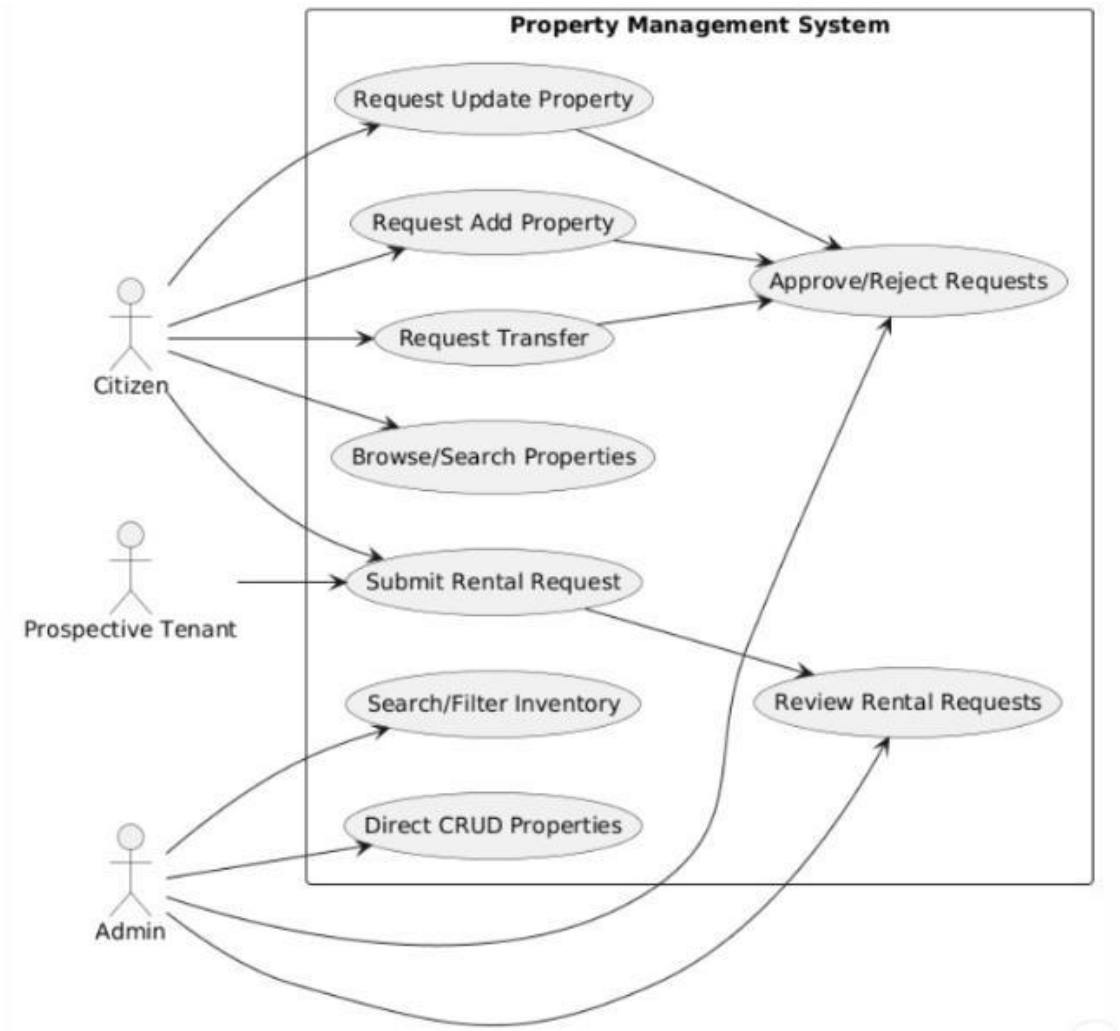


Fig 3: Use case diagram — Property Management module

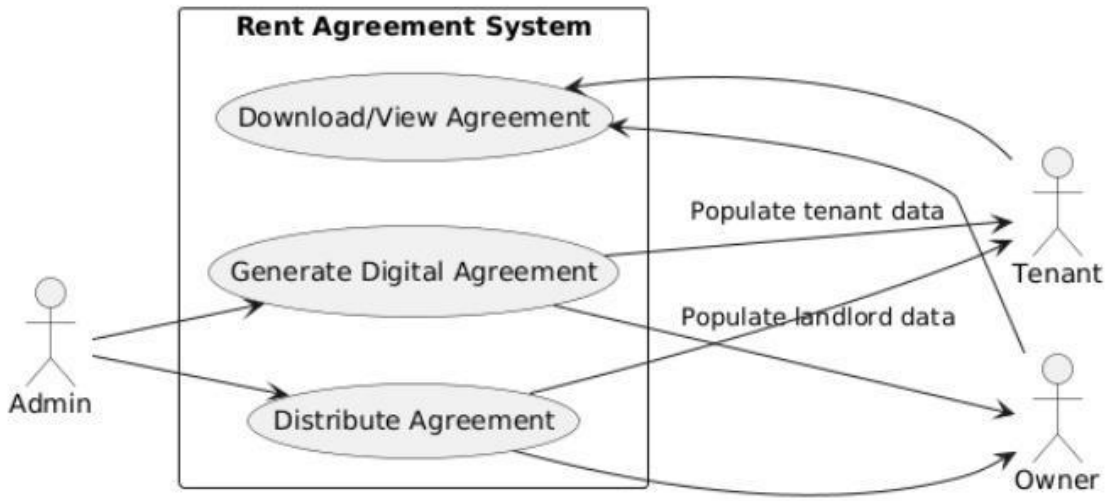


Fig 4: Use case diagram — Rent Agreement module

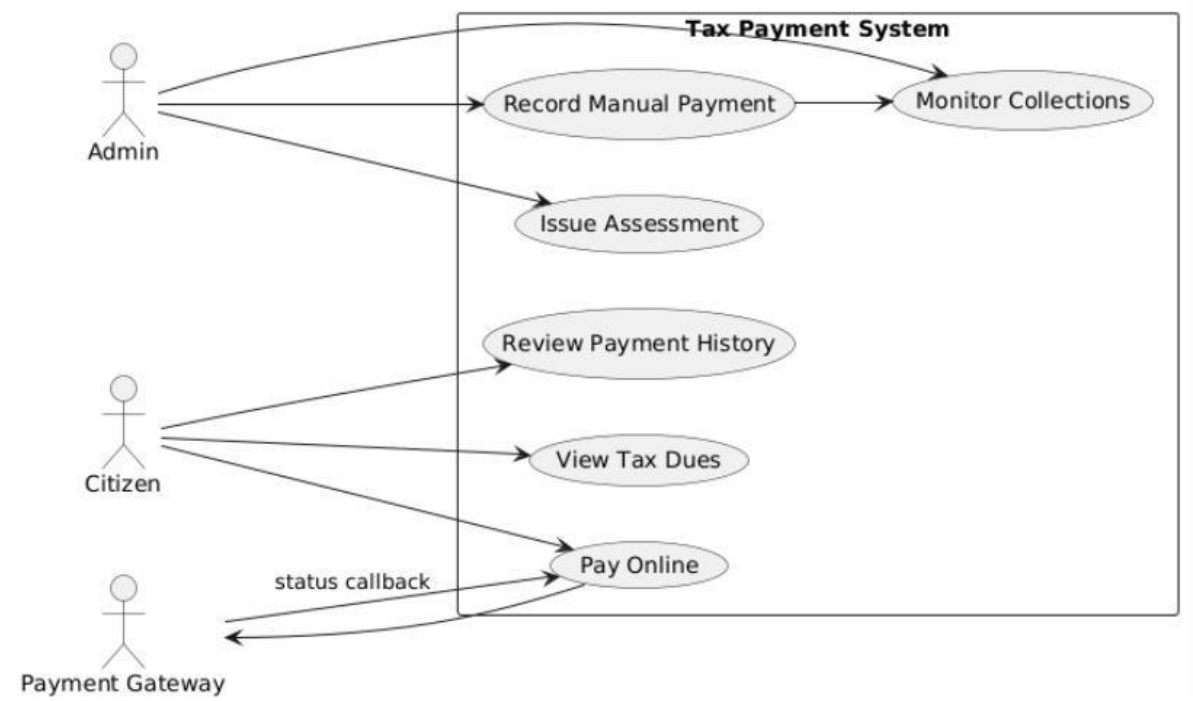


Fig 5: Use case diagram — Tax Payment module

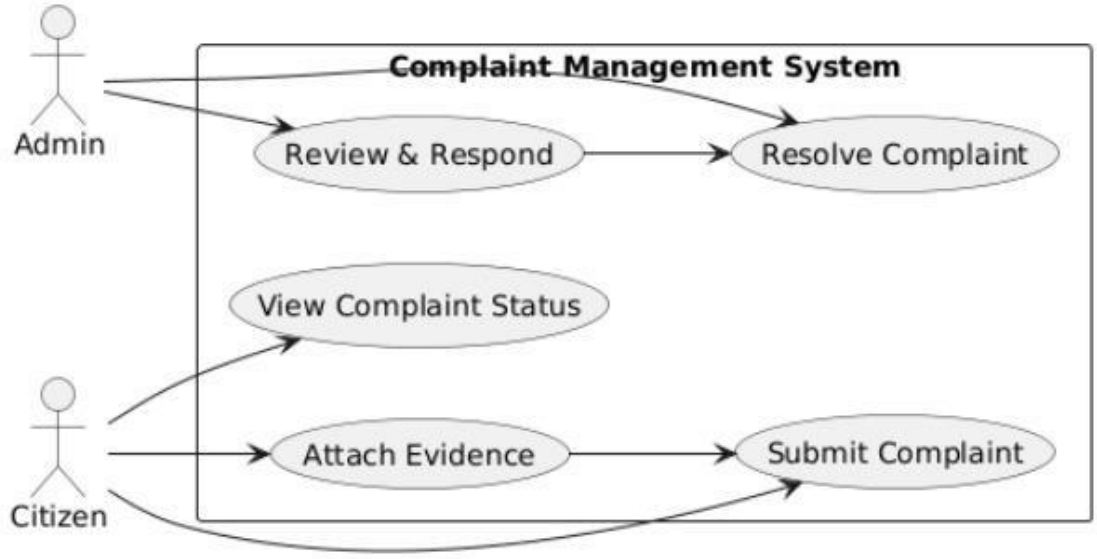


Fig 6: Use case diagram — Complaint Management module

Sequence Diagram

The sequence diagram traces a representative end-to-end flow: login and role routing, citizen verification submission and admin approval, a property/tax assessment being generated, a Stripe-hosted checkout payment with webhook confirmation, a complaint being filed and resolved, and an admin pulling revenue KPIs.

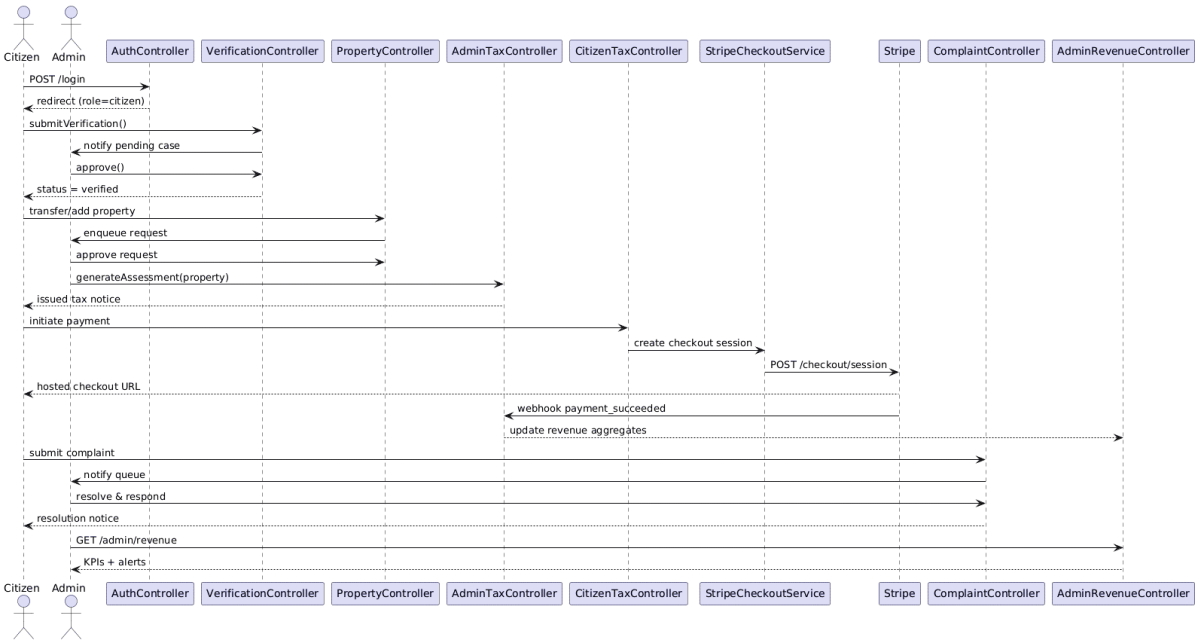


Fig 7: System sequence diagram across modules

Activity and State Diagrams

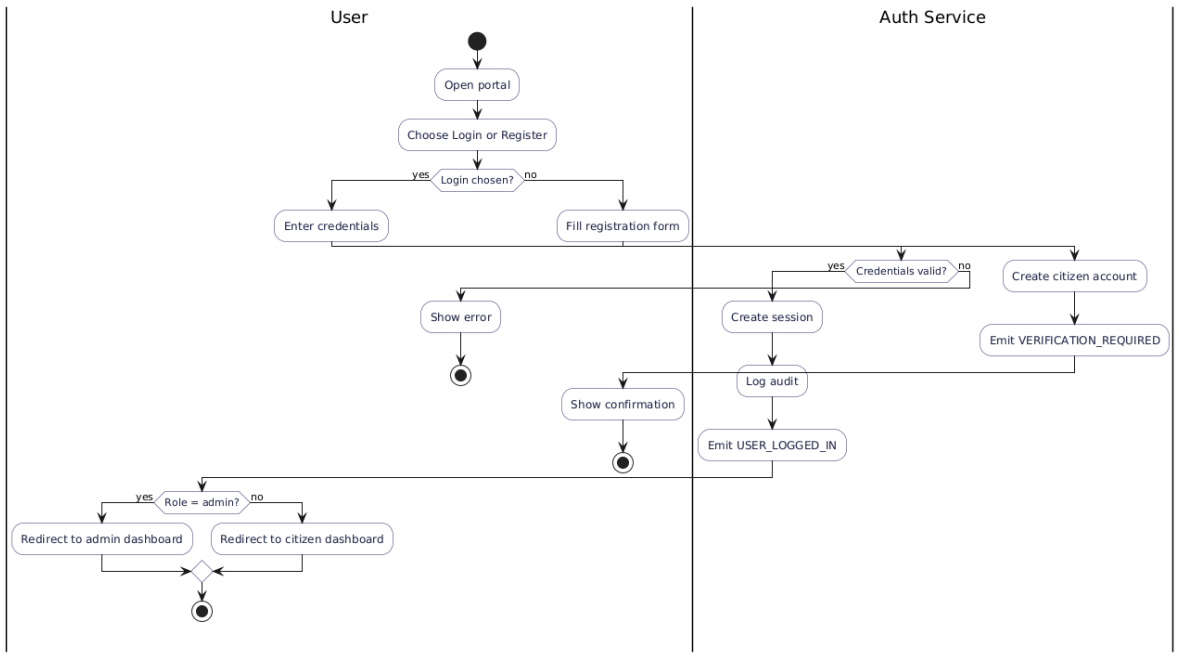


Fig 8: Activity diagram — Authentication (login/registration and role routing)

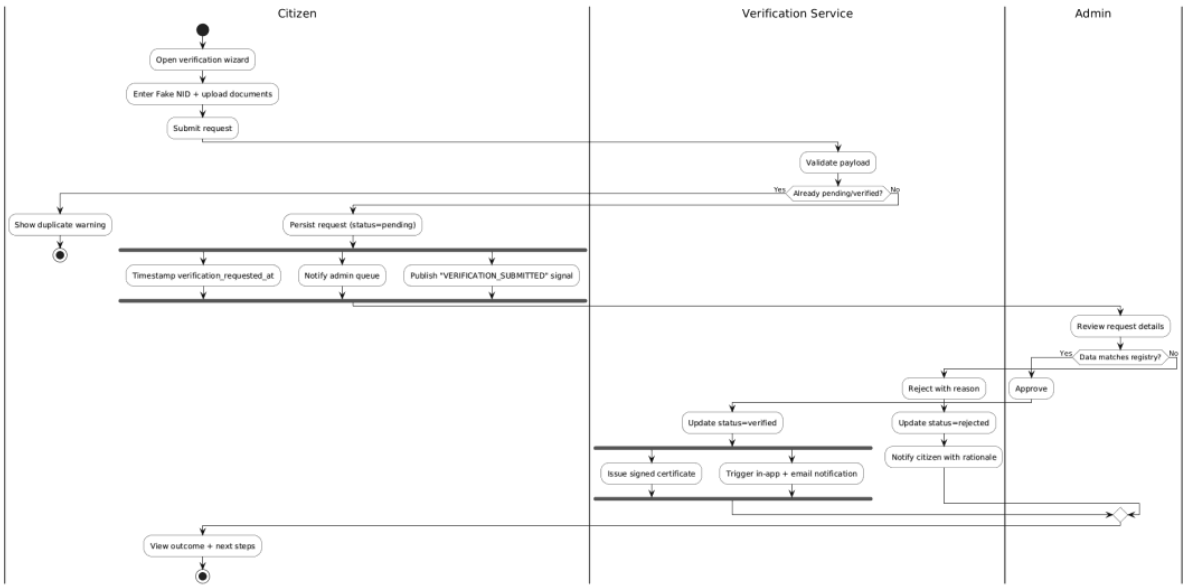


Fig 9: Activity diagram — Citizen Verification workflow

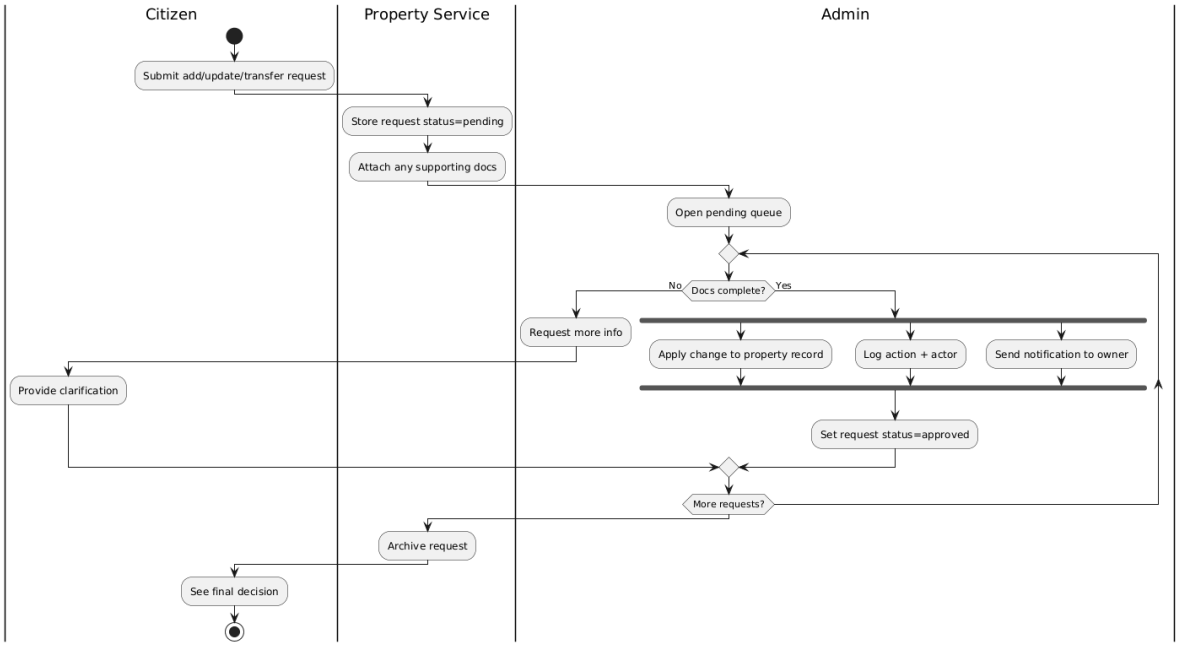


Fig 10: Activity diagram — Property Management (add/update/transfer approval)

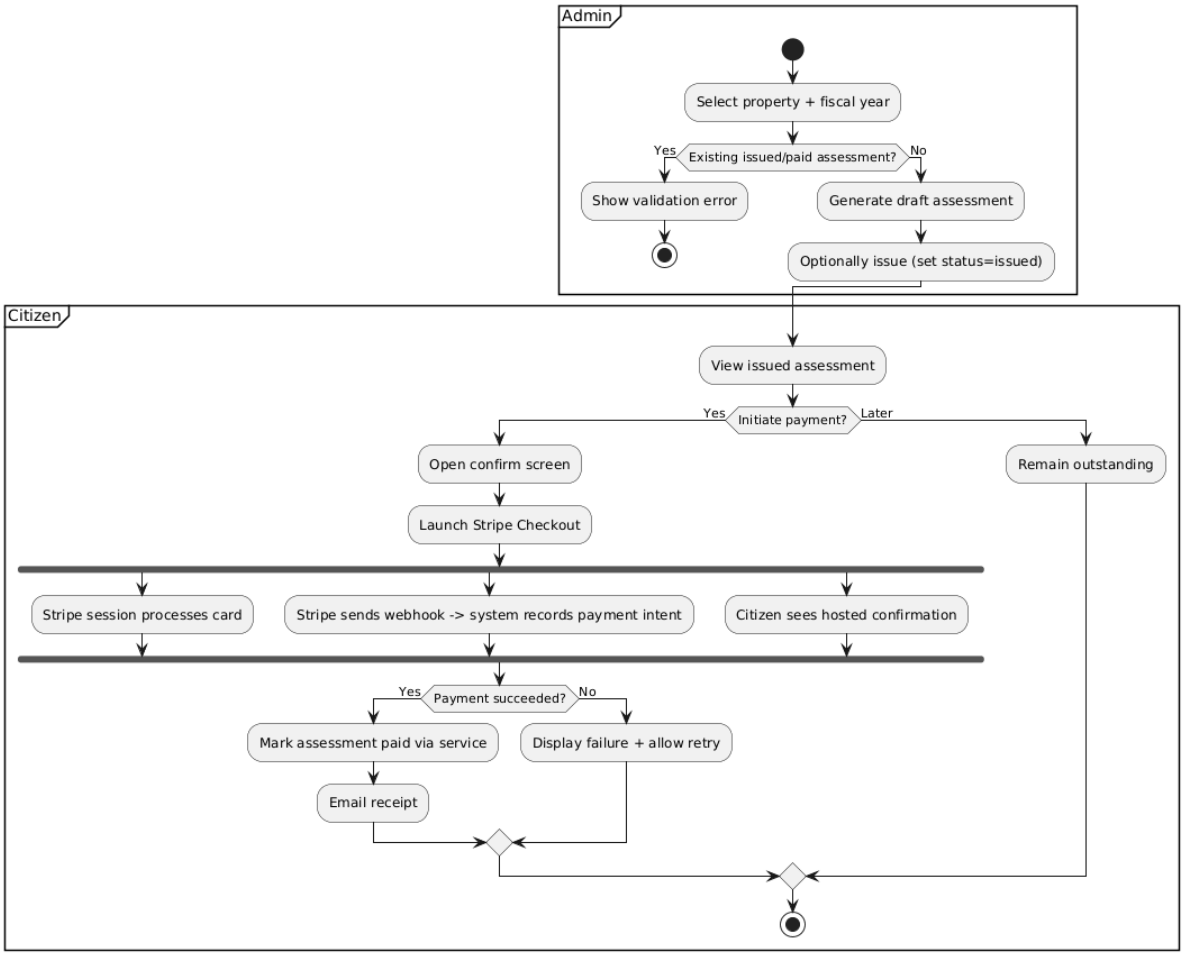


Fig 11: Activity diagram — Tax Payment workflow (assessment to Stripe checkout to receipt)

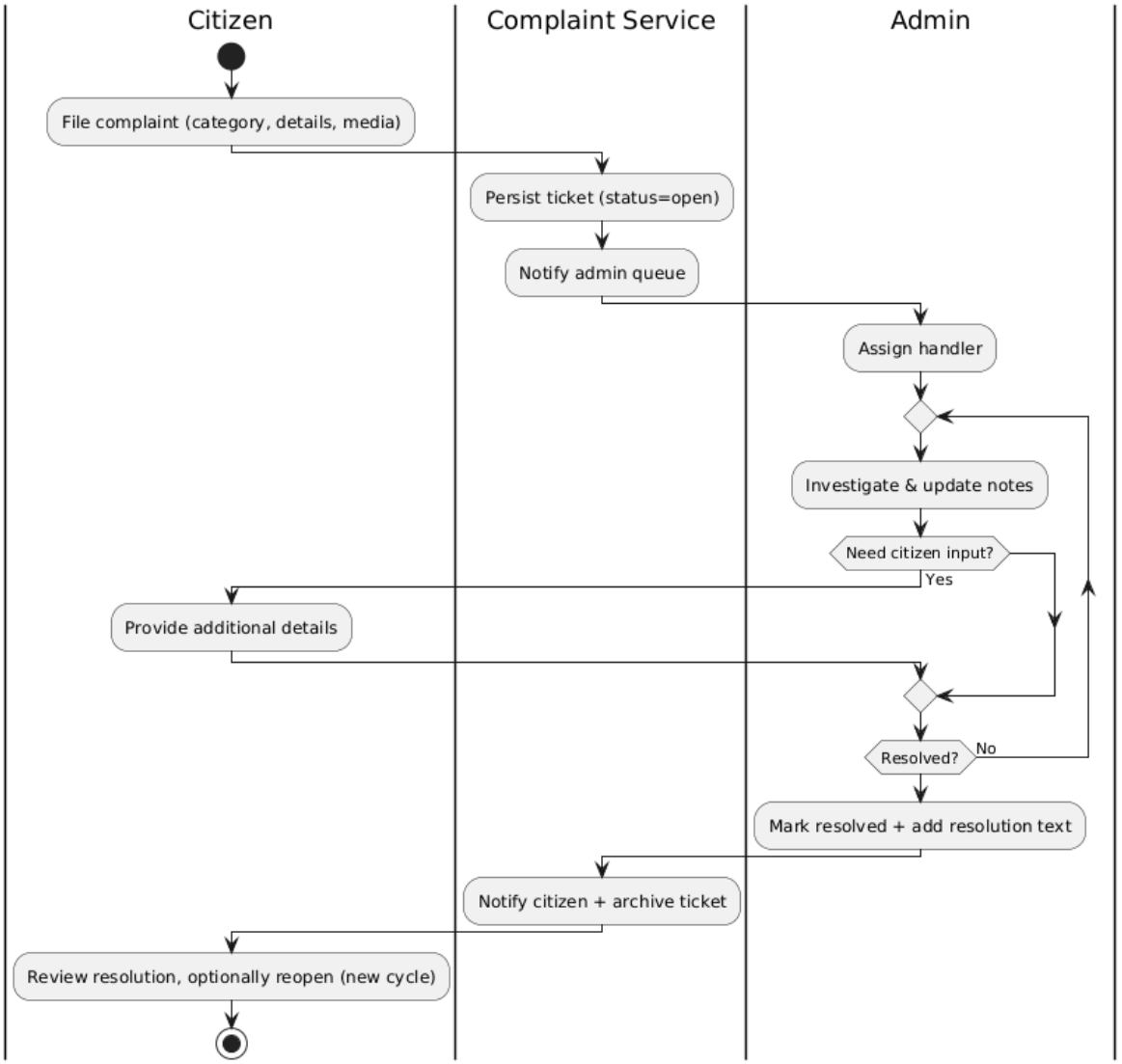


Fig 12: Activity diagram — Complaint Management workflow

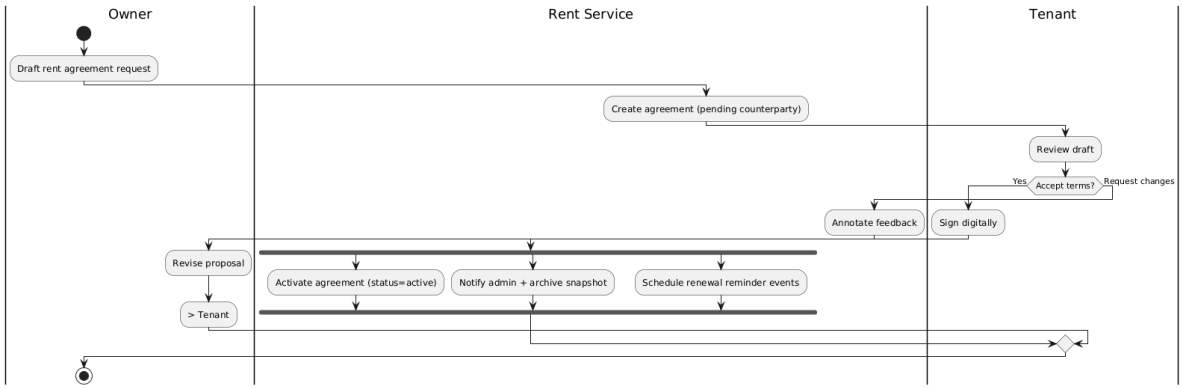


Fig 13: Activity diagram — Rent Agreement workflow

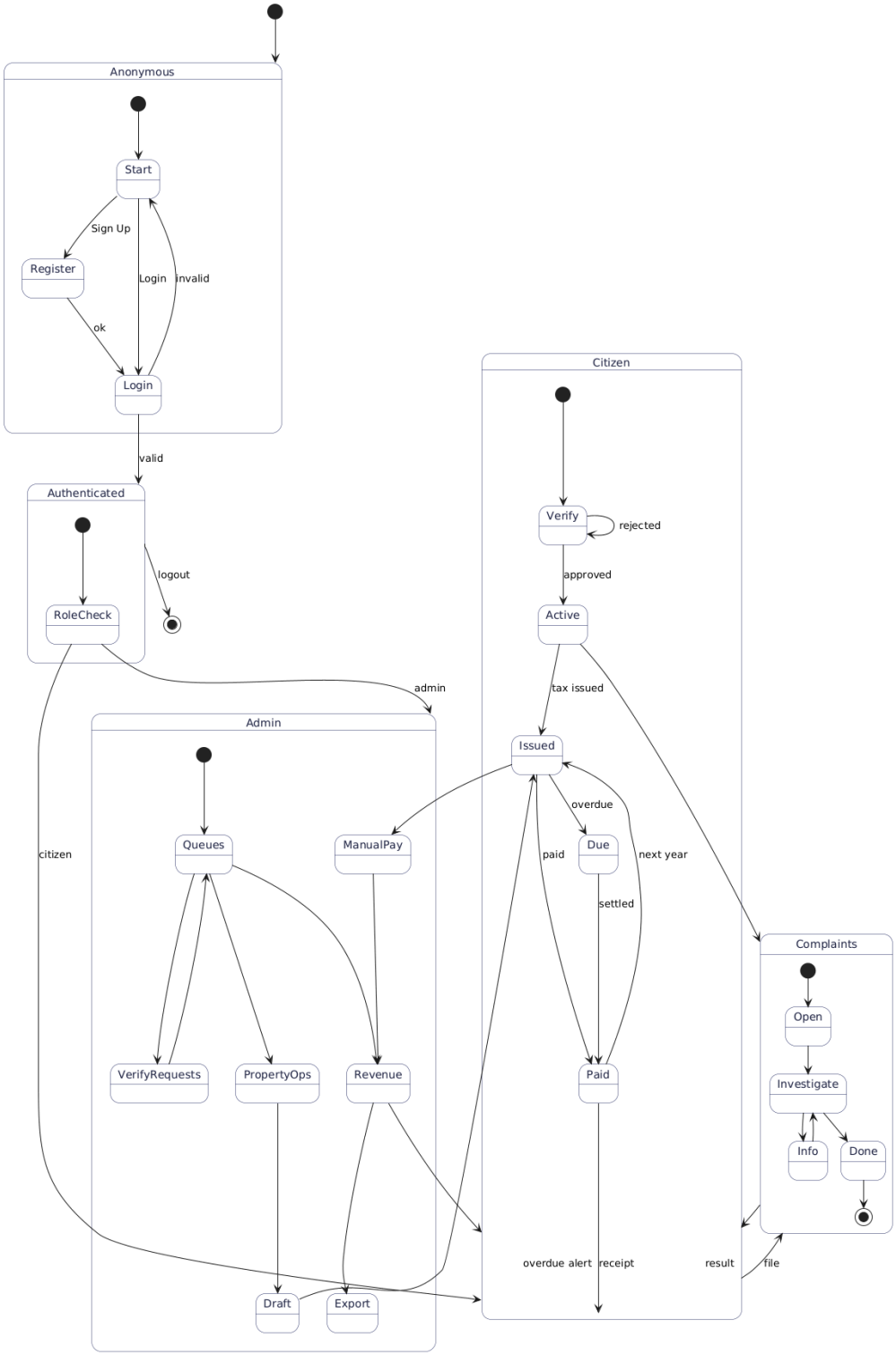


Fig 14: State diagram — overall system states across authentication, citizen, admin, and complaint lifecycles

Class and Class Collaboration Diagram

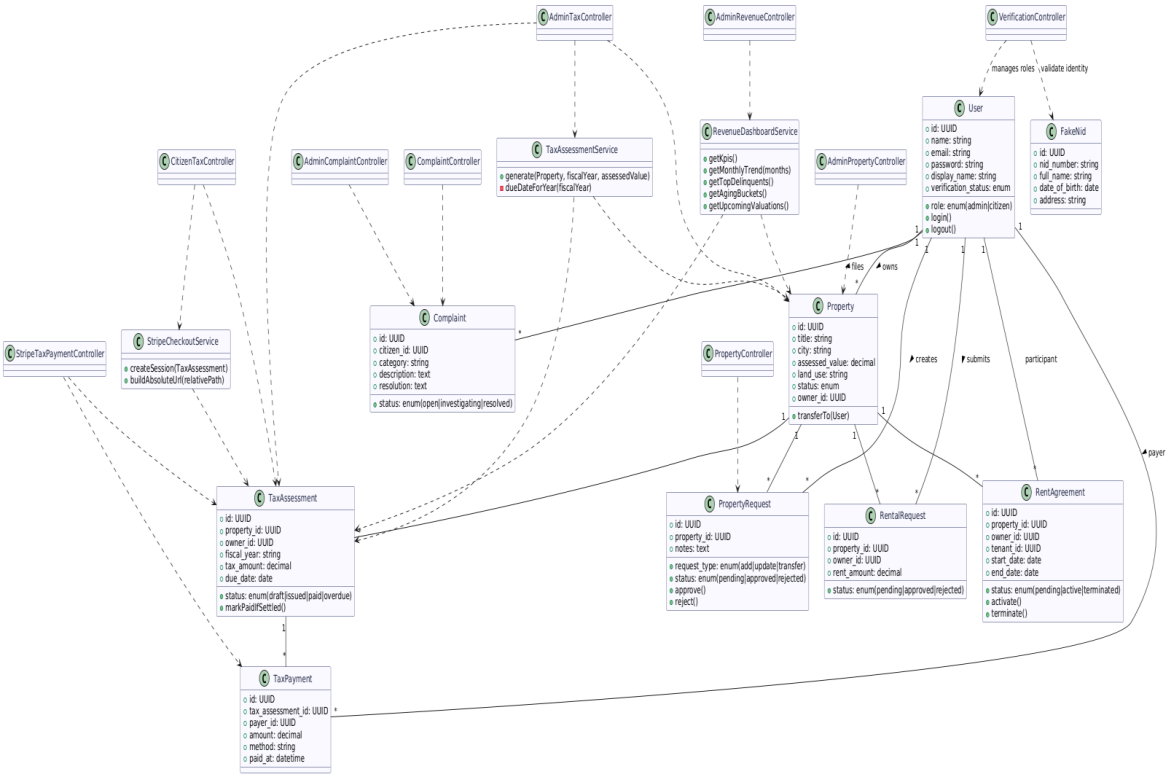


Fig 15: Class collaboration diagram — entities (User, Property, Property Request, RentalRequest, Rent Agreement, Tax Assessment, Tax Payment, Complaint, Fake Nid), services, and controllers

Data Model Summary

The PostgreSQL schema (managed through EF Core's ApplicationDbContext) centers on nine entities:

- User — citizen/admin account with role, verification status, NID fields, and uploaded document paths.
- FakeNid — the reference NID dataset used to cross-check submitted verification details.
- Property — a registered property with owner, valuation, land use, and rent-availability fields.
- PropertyRequest — a pending add/update/transfer request awaiting admin decision.
- RentalRequest — a tenant's rental application, including tenant- and owner-proposed terms.
- RentAgreement — the generated agreement once a rental request is approved (status: pending/active/terminated).
- TaxAssessment — a per-property, per-fiscal-year tax record (status: draft/issued/paid/overdue).
- TaxPayment — a recorded payment (card, bank transfer, bKash, Nagad, or office/cash) against an assessment.
- Complaint — a citizen-filed ticket with category, status, and admin reply.

Security and Access Control

Authentication uses ASP.NET Core's cookie authentication scheme with a sliding 8-hour session, and passwords are hashed using ASP.NET Core's built-in IPasswordHasher. Authorization is role-based (citizen vs. admin), enforced at the controller/action level so that citizen-only and admin-only routes are separated, in line with the role-based access control specified in the requirements.

Implementation

Development Process

Implementation proceeded module by module, matching the Agile sprints defined in the methodology: each module's EF Core entity/entities were defined first, followed by the controller actions, the supporting service (where business logic was non-trivial, e.g., tax calculation or Stripe session creation), and finally the Razor views. The database schema was created using EF Core's EnsureCreated() on first run (with a PostgresSchemaManager repair step for compatibility with pre-existing databases), and demo data (an admin and a citizen account, plus sample properties/assessments) is seeded automatically via DbInitializer so the system is immediately usable after deployment.

Key Features and Functionalities

1. Authentication Module

- Citizen self-registration and login; a separate admin login.
- Registration is gated by a 6-digit email OTP (one-time password), valid for 15 minutes and dispatched via the Gmail API (EmailService), before the account can log in.
- Cookie-based session authentication with role claims (citizen / admin).
- Role-based route protection across every other module.

2. Citizen Verification Module

- Citizens submit NID number, personal details, and NID front/back plus a passport photo.
- Submitted details are cross-checked against the FakeNid reference dataset (auto-fill from registry and field-by-field comparison).
- Admins review, approve, or reject; verified citizens receive a status update and certificate.

3. Property Management Module

- Citizens submit add / update / transfer property requests, which are stored as PropertyRequest records pending admin decision.
- On approval, the system creates (or updates) the real Property record and automatically generates the first tax assessment for it.
- Citizens can browse and filter community properties, and view their own registered properties.

4. Rent Agreement Module

- Prospective tenants submit a RentalRequest with proposed terms; owners can counter-propose terms and mark the request ready for admin review.
- On admin approval, a RentAgreement is generated and the property is marked unavailable for further rental.
- Landlords and tenants can view/download their agreement.

5. Tax Payment Module

- Admins generate and issue per-fiscal-year TaxAssessment records using a configurable tax rate (default 1%, due June 30 — see appsettings.json Tax section).
- Citizens view outstanding dues and payment history, and pay via Stripe Checkout (card) or the demo/manual bank transfer, bKash, and Nagad options.
- A background overdue-flagging routine (OverdueAssessmentService) marks unpaid assessments overdue after their due date, running as a scheduled background job.

6. Complaint Management Module

- Citizens file complaints with a category, description, optional property link, and optional attachment (JPG/PNG/PDF, up to 5 MB).
- Admins review, respond, and resolve/reject complaints; citizens can track status and read the official reply.

Code Structure and Major Modules

The solution is organized as a single ASP.NET Core MVC project with a conventional folder layout:

Folder	Contents
Controllers/	15 MVC controllers (Auth, Base, Citizen, Admin — Dashboard/Property/RentAgreement/Revenue/Tax/Complaint, CitizenTax, CitizenRentAgreement, Property, Verification, Complaint, FakeNidStatus)
Models/	9 EF Core entity classes
Services/	6 business-logic services — TaxAssessmentService, RevenueDashboardService, StripeCheckoutService, NidVerificationService, OverdueAssessmentService, and EmailService (Gmail-API OTP/email delivery)
Data/	ApplicationDbContext, DbInitializer (seed data), PostgresSchemaManager
Views/	Razor views grouped by area: auth, citizen (properties, taxes, complaints, rent-agreements, profile), admin (verification, properties, taxes, revenue, complaints, rent-agreements, citizens), dashboard, layouts, Shared
wwwroot/	Static assets — CSS design system (site.css, professional.css), site.js, and uploaded avatars

Configuration

Runtime configuration lives in appsettings.json: the PostgreSQL connection string, Stripe settings (secret key, currency, success/cancel URLs, and a DemoMode flag that lets the payment flow work without real Stripe credentials), and the default tax rate/due date. Two seeded demo accounts are provided out of the box — admin@citizen.local / Admin@123 and citizen@citizen.local / Citizen@123 — to support grading and demonstration without manual data entry.

Results and Discussion

Project Achievements

All six planned modules — Authentication, Citizen Verification, Property Management, Rent Agreement, Tax Payment, and Complaint Management — were implemented and are reachable end-to-end through the routes documented in the project's workflow notes (for example, /verification/apply, /citizen/properties/create, /citizen/taxes/{id}/pay, and /admin/revenue). The property-approval, rental-approval, and tax-assessment flows correctly chain together: an approved property request creates a real Property and its first TaxAssessment automatically, and an approved rental request creates an active RentAgreement while marking the property unavailable for further rent, matching the objectives set out in the Introduction.

Manual Workflow Verification

The team's verification for this final version was carried out as structured manual walkthroughs of each citizen and admin flow (registration → verification → property request → approval → tax issuance → payment → complaint → resolution) rather than through an automated test suite; no automated test project has yet been created for the codebase. Each module in the Implementation section above was exercised against the two seeded demo accounts to confirm the expected status transitions (e.g., PropertyRequest: pending → approved, RentalRequest: pending → approved → RentAgreement: active, TaxAssessment: draft → issued → paid).

Discussion of Challenges

- Framework depth: building out the full ASP.NET Core/EF Core/Razor stack across every controller, model, and view required substantial implementation effort and was the single largest source of work in the project.
- Payment integration: supporting both a real Stripe flow and a credential-free demo flow (Card/Bank/bKash/Nagad) required careful separation of the payment-recording logic from the payment-collection UI so that grading/demo use does not require live Stripe keys.
- Approval-driven state machines: property, rental, and tax records all move through multi-step admin-approval states, which required careful server-side validation to prevent citizens from bypassing pending states (e.g., paying an assessment that is not yet issued, or renting a property that is already occupied).

Comparison with Initial Objectives

Measured against the objectives in the Introduction, the delivered system meets all core functional goals: secure role-based authentication, digitized NID verification, approval-gated property and rent-agreement management, an online/demo tax-payment workflow, and a structured complaint channel. The main limitation relative to the original plan is that the project does not yet include an automated test suite.

Conclusion and Future Work

Summary of Findings

The Citizen Management project demonstrates that a modular, role-secured MVC application can successfully replace paper-based municipal workflows for identity verification, property management, rent agreements, tax collection, and complaint handling. Rebuilding the system on ASP.NET Core MVC with Entity Framework Core and PostgreSQL preserved the module boundaries and workflows defined during the original design phase while giving the team a strongly-typed, actively maintained framework to build on.

Project Limitations

- Citizen identity verification relies on a local reference NID dataset (FakeNid) rather than a live national ID registry.
- The Stripe payment integration runs in demo mode by default; a production deployment would need a verified Stripe (or local payment gateway) account and PCI-relevant hardening.
- There is currently no automated unit/feature test project for the ASP.NET Core codebase, so regression testing is manual.
- The system assumes a single municipal entity and two user roles; multi-tenant or multi-role (e.g., landlord-only) support was out of scope.
- The database uses EF Core's EnsureCreated() rather than versioned migrations, which is convenient for coursework but not recommended for a production rollout.

Suggestions for Future Development

- Integrate with a real national ID / e-Government registry for automated, authoritative identity verification.
- Introduce an automated test project (xUnit/NUnit) covering the controllers and services, replacing the manual verification used for this release.
- Move from EnsureCreated() to versioned EF Core migrations for safer schema evolution in production.
- Add two-factor authentication and structured audit logging for admin actions.
- Build a companion mobile application and add SMS/email notifications for status changes.
- Expand payment options with live bKash/Nagad APIs and multi-gateway support.
- Add analytics dashboards (trend analysis for complaints, revenue forecasting) building on the existing RevenueDashboardService.

Appendix

Expected Deliverables (per Project Proposal)

- Fully functional source code repository: a production-grade ASP.NET Core application hosted on GitHub.
- Comprehensive database schema: a fully normalized relational database mapping citizen identities and municipal transactions, delivered on PostgreSQL.
- Comprehensive SQA documentation: unit testing suites verifying role boundaries, middleware safety, and webhook events — not yet delivered for the ASP.NET Core codebase (see Limitations).
- Final project report: architectural diagrams, endpoint specifications, and a guide for future biometric/mobile expansion — this document.

Demo Access

- Demo Admin Login: admin@citizen.local / Admin@123
- Demo Citizen Login: citizen@citizen.local / Citizen@123